Understanding the Task Scoring Algorithm

Overview of the Algorithm

The algorithm is designed to **rank tasks by importance**, not just by priority alone.

It works by:

1. Assigning a **numerical score** to each task
2. Combining multiple factors into that score
3. Sorting tasks from **highest score → lowest score**

This approach ensures that the most urgent and relevant tasks appear first.

Key Components of the Algorithm

Base Priority Score

Each task starts with a score based on its priority:

LOW → 10
MEDIUM → 20
HIGH → 30
URGENT → 40

Insight: Priority is important, but not the only factor.

Due Date Adjustment

Tasks get additional points depending on urgency:

Overdue → +30
Due Today → +20
Due in ≤ 2 days → +15
Due in ≤ 7 days → +10

📌 Insight: A low-priority task can outrank a high-priority one if it's overdue.

2.3 Status Penalty

DONE → -50
REVIEW → -15
Insight: Completed tasks are intentionally pushed to the bottom.

2.4 Tag-Based Boost

If a task contains any of these tags:

"blocker", "critical", "urgent"

It gets:

+8 points

Insight: Tags represent **real-world importance** that priority alone may miss.

Visual Diagram of Scoring Flow

```
+----------------------+
|     Task Input       |
+----------+-----------+
           |
           v
+---------------------------+
| Priority Score (Base)     |
+---------------------------+
           |
           v
+---------------------------+
| Due Date Adjustment       |
+---------------------------+
           |
           v
+---------------------------+
| Status Penalty            |
+---------------------------+
           |
           v
+---------------------------+
| Tag Boost                 |
+---------------------------+
           |
           v
+---------------------------+
| Recent Update Boost       |
+---------------------------+
           |
```

```
        v
+----------------------+
|   Final Score        |
+----------------------+
        |
        v
+----------------------+
| Sort Tasks (DESC)    |
+----------------------+
```

Sorting Logic

The algorithm sorts tasks using:

Highest Score → Lowest Score

Visual Example:

Task A → 58
Task B → 42
Task C → 10

Sorted:
1. Task A
2. Task B
3. Task C

Core Technique Used

Weighted Scoring System

This algorithm uses a **weighted scoring model**, where:

- Different factors contribute differently
- Some factors increase importance (boosts)
- Some reduce importance (penalties)

This is commonly used in:

- Task management systems
- Recommendation engines
- Ranking systems

Key Insights & Learning Points

Priority is Not Everything

Even a LOW priority task can rank higher if:

- It is overdue
- It has critical tags

Combining Multiple Factors

The algorithm shows how to:

- Convert real-world conditions into numbers
- Combine them into one decision metric

Use of Boosts and Penalties

- Boosts → increase urgency
- Penalties → reduce importance

This creates a balanced scoring system.

Efficiency Consideration

The score is recalculated during sorting:

calculateTaskScore(b) - calculateTaskScore(a)

Learning:

- This can be inefficient for large datasets
- A better approach is to **precompute scores**

Insight 6: Real-World Relevance

This algorithm reflects how humans prioritize tasks:

- Urgency (due date)
- Importance (priority)
- Context (tags)
- Progress (status)
- Activity (recent updates)

Final Understanding

The algorithm is not just sorting tasks — it is:

Simulating human decision-making using a scoring system

It transforms multiple task attributes into a single value, making it easy to:

- Compare tasks
- Rank tasks
- Select top priorities

Conclusion

Through this algorithm, I learned:

- How to design a **multi-factor ranking system**
- How to translate logic into numerical scoring
- How sorting can be enhanced using computed values instead of raw data

Personal Reflection

This helped me understand that:

- Good systems don’t rely on one variable
- Real-world problems require combining multiple signals
- Small design choices (like +5 or -50) can significantly affect outcomes